

# GS2026.1

Inteligência Artificial Generativa

## Documento Técnico

Assistente Inteligente com RAG para Dados e Documentos Espaciais

|             |                                                                 |
|-------------|-----------------------------------------------------------------|
| Projeto     | Assistente RAG — Nova Economia Espacial                         |
| Arquitetura | Retrieval-Augmented Generation (RAG)                            |
| Domínios    | Clima · Satélites · Agricultura · Ambiente · Desastres · Espaço |
| Stack       | Python · Streamlit · ChromaDB · sentence-transformers · LLM     |

# 1. Problema escolhido

A nova economia espacial gera um volume crescente de dados e documentos técnicos — relatórios de agências como NASA e ESA, boletins climáticos, artigos sobre agricultura de precisão e laudos de monitoramento ambiental. Esse conteúdo costuma estar disperso em arquivos longos (PDF, TXT, DOCX), o que torna difícil e demorado localizar uma informação específica.

O problema escolhido é **facilitar a consulta a esse acervo**: permitir que um usuário faça perguntas em linguagem natural e receba respostas fundamentadas exclusivamente nos documentos fornecidos, cobrindo os temas clima, satélites, agricultura inteligente, monitoramento ambiental, desastres naturais e exploração espacial.

A solução adotada é um **assistente baseado em RAG (Retrieval-Augmented Generation)**. Em vez de depender apenas do conhecimento interno do modelo de linguagem — sujeito a desatualização e a alucinações —, o sistema primeiro **recupera** os trechos mais relevantes dos documentos e só então **gera** a resposta com base nesse contexto, reduzindo erros e permitindo citar as fontes.

# 2. Arquitetura da solução

A aplicação é dividida em duas camadas independentes: a **interface** (app.py, em Streamlit) e o **motor RAG** (rag\_engine.py), que concentra toda a lógica de ingestão, indexação, recuperação e geração. Essa separação facilita testes e manutenção.

## Diagrama do pipeline



Figura 1 — Fluxo de dados da arquitetura RAG.

# 3. Ferramentas utilizadas

| Camada    | Ferramenta | Função                                    |
|-----------|------------|-------------------------------------------|
| Interface | Streamlit  | Upload de documentos e chat com o usuário |

|              |                       |                                    |
|--------------|-----------------------|------------------------------------|
| Leitura PDF  | pypdf                 | Extração de texto de arquivos PDF  |
| Leitura DOCX | python-docx           | Extração de texto de arquivos Word |
| Leitura TXT  | Python nativo         | Leitura de texto puro              |
| Embeddings   | sentence-transformers | Conversão de texto em vetores      |
| Vector store | ChromaDB              | Indexação e busca por similaridade |
| Geração      | openai (cliente)      | Chamada ao LLM (Groq/OpenAI)       |

## 4. Modelo utilizado

### Modelo de embeddings

**sentence-transformers/all-MiniLM-L6-v2** — modelo compacto (~90 MB) que transforma cada trecho de texto em um vetor de 384 dimensões. É executado **localmente e sem custo**, gerando os embeddings tanto dos documentos quanto das perguntas. Os vetores são normalizados para uso de similaridade de cosseno.

### Modelo generativo (LLM)

A geração da resposta usa um **LLM acessado por API compatível com a OpenAI**. A configuração padrão aponta para o **Groq** com o modelo **llama-3.3-70b-versatile** (possui plano gratuito); alternativamente, pode-se usar a OpenAI com **gpt-4o-mini**. O desacoplamento por meio de uma API compatível permite trocar o provedor sem alterar o código.

## 5. Vector store utilizada

Foi utilizado o **ChromaDB** em modo persistente (PersistentClient), o que mantém o índice em disco entre execuções. Cada trecho é armazenado com seu vetor, o texto original e metadados (arquivo de origem e número do trecho), permitindo citar a fonte de cada resposta. A coleção é configurada com distância de **cosseno** (hnsw:space=cosine), adequada a embeddings normalizados.

## 6. Fluxo RAG implementado

- **Ingestão:** o documento é lido e o texto extraído conforme o formato.
- **Chunking:** o texto é dividido em trechos de 800 caracteres com 120 de sobreposição, evitando cortar ideias entre trechos.
- **Embeddings:** cada trecho é vetorizado pelo all-MiniLM-L6-v2.
- **Indexação:** os vetores e metadados são gravados no ChromaDB.
- **Recuperação:** a pergunta é vetorizada e os 4 trechos mais similares são recuperados por distância de cosseno.
- **Geração:** os trechos compõem o contexto de um prompt enviado ao LLM, instruído a responder apenas com base no contexto e a admitir quando a informação não estiver presente.
- **Resposta:** o texto gerado é exibido junto das fontes consultadas.

## 7. Limitações da solução

- A resposta depende do acervo indexado; o RAG não cria informação ausente nas fontes.
- O modelo de embeddings, embora multilíngue, é otimizado para inglês; termos muito técnicos em português podem reduzir a precisão da busca.
- Não há OCR: PDFs escaneados (imagem) não têm texto extraível diretamente.
- O chunking é por contagem de caracteres e não respeita fronteiras semânticas de forma sofisticada.
- A geração final depende de um serviço externo (API do LLM) e de conexão com a internet.

## 8. Melhorias futuras

---

- Chunking semântico (por parágrafo/sentença) e re-ranking dos trechos recuperados.
- Suporte a OCR para PDFs escaneados.
- Citação com número de página e destaque do trecho exato na resposta.
- Memória de conversa para perguntas de acompanhamento.
- Avaliação automática da qualidade das respostas (ex.: framework RAGAS).
- Implantação em nuvem (Streamlit Community Cloud ou Hugging Face Spaces).

---

Documento técnico gerado para o projeto GS2026.1 — Inteligência Artificial Generativa. Código-fonte e instruções completas no repositório do projeto (ver README.md).